

PTVA'S
MULUND COLLEGE OF COMMERCE (AUTONOMOUS)
DEPARTMENT OF INFORMATION TECHNOLOGY
T.Y. B.Sc. Information Technology

Name: Sawant Aarush Hemant

Roll No: 269729

SEM: V

Course Name: Artificial Intelligence – Practical

INDEX

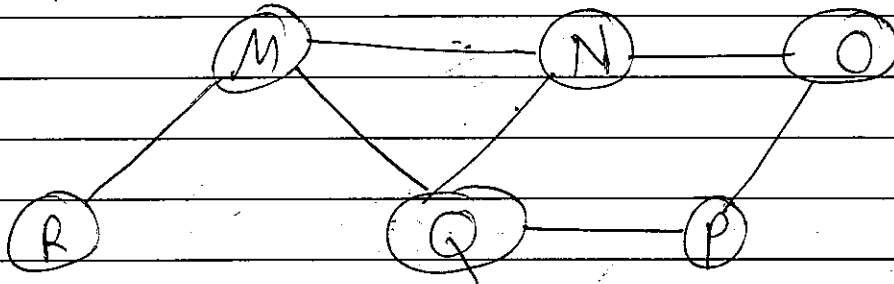
PR. NO	PRACTICAL TITLE	DATE	SIGNATURE (Attendance)	SIGNATURE (Submission)
1.	a. Depth First Search b. Breadth First Search	22/06/2026	<u>Qm</u> 22/6	<u>Qm</u> 29/06
2.	a. N-Queen Problem b. Tower of Hanoi	13/07/2026 21/07/2026	<u>Qm</u> 13/7	<u>Qm</u> 21/7
3.	a. Alpha Beta Search b. Hill Climbing			
4.	a. A* Algorithm b. Greedy Best First Search	29/06/2026 13/07/2026	<u>Qm</u> 29/6 <u>Qm</u> 13/7	<u>Qm</u> 29/6 <u>Qm</u> 13/7
5.	a. Water Jug Problem b. Travelling Salesman Problem	11/08/2026	<u>Qm</u> 11/8	
6.	a. Missionaries and Cannibals b. Number Puzzle			
7.	a. Shuffle Deck of cards b. Tic-Tac-Toe game	31/07/2026	<u>Qm</u> 31/7	<u>Qm</u> 11/08
8.	a. Block of Word Problem b. Constraint Satisfaction Problem			
9.	Derive the expression based on a. Associative law b. Distributed law	21/07/2026	<u>Qm</u> 21/7	<u>Qm</u> 31/7
10.	a. Derive the predicate b. Given predicates – i. Draw family tree; ii. Definition			

Final Submission: _____

22/06/26

Practical - 1

a) Depth First Search



```

graph = {
    'M' : ['N', 'Q', 'R'],
    'N' : ['O', 'Q', 'M'],
    'R' : ['M'],
    'O' : ['N', 'P'],
    'Q' : ['M', 'N', 'P'],
}
  
```

```

def dfs(g, n, seen, d):
    if n not in seen:
        seen.append(n)
        for i in g[n]:
            if seen[-1] is d:
                break
            dfs(g, i, seen, d)
    return seen
  
```

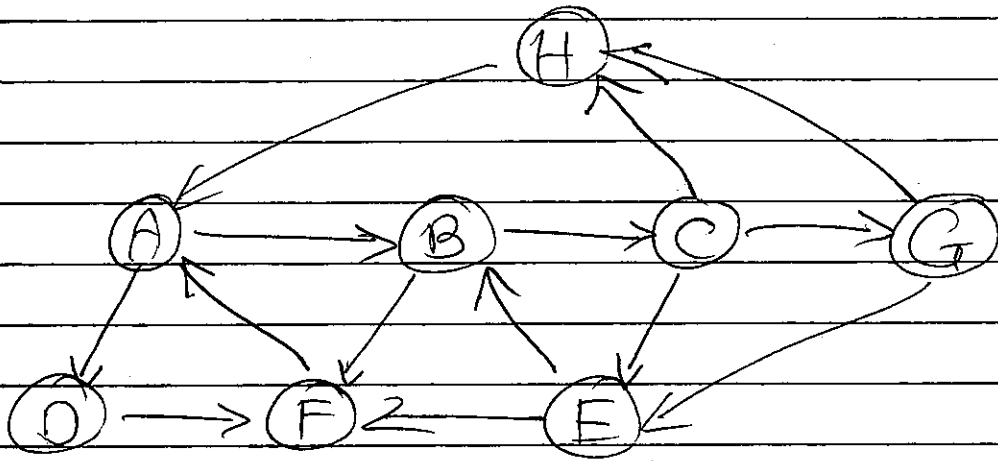
```

print(dfs(graph, 'M', [], 'O'))
  
```

Output:-

```
['M', 'N', 'O']
```

b) Breadth First Search



```
import collections
```

```
def bfs(graph, root):  
    seen, queue = set([root]), collections.deque([root])  
    while queue:  
        vertex = queue.popleft()  
        visit(vertex)  
        for node in graph[vertex]:  
            if node not in seen:  
                seen.add(node)  
                queue.append(node)
```

all paths

```
def allpath(st, end, gr):  
    todo = [(st, [st])]   
    while len(todo):  
        node, path = todo.pop(0)  
        for next_node in gr[node]:  
            if next_node in path:  
                continue
```

```
print("ideal solution")
elif next_node == end:
    yield path + [next_node]
```

```
else:
    todo.append((next_node, path +
```

```
[next_node]))
```

```
def visit(n):
    print(n)
```

```
def bfs_shortest_path(graph, source, destination,
```

```
    checked = []
```

```
    queue = [source]
```

```
    if source == destination:
```

```
        return "SOURCE IS DESTINATION:"
```

```
    while queue:
```

```
        path = queue.pop(0)
```

```
        node = path[-1]
```

```
        if node not in checked:
```

```
            neighbours = graph[node]
```

```
            for neighbour in neighbours:
```

```
                new_path = list(path)
```

```
                new_path.append(neighbour)
```

```
                queue.append(new_path)
```

```
                if neighbour == destination:
```

```
                    return new_path
```

```

        'checked', append (node)
    return "PATH DOES NOT EXIST:"

```

```

graph = {
    'A': ['B', 'D'],
    'B': ['C', 'F'],
    'C': ['E', 'G'],
    'G': ['E'],
    'E': ['B', 'F'],
    'F': ['A'],
    'D': ['F'],
}

```

```

print("GRAPH TRAVERSAL")
bfs(graph, 'A')
print("\n\nall path is:")
[print(x) for x in allpath('A', 'E', graph)]
print("\n\nSHORTEST PATH OF GRAPH IS:")
bfs-shortest-path(graph, 'A', 'E')

```

Output :-

GRAPH TRAVERSAL

A

B

D

E

F

E

G

all path is:

['A', 'B', 'C', 'E']

['A', 'B', 'C', 'G', 'E']

SHORTEST PATH OF
GRAPH IS: ['A', 'B', 'C', 'E']

Sr.No. Date

2

13.21/7/26

Topic

N-Queen Problem & Tower of Hanoi

Sign

em

1) N Queen Problem:

Herein, we've to place 8 queens in a 8×8 chess board, so that no queen touches each other

Code :-

```
def print_board(board):  
    for row in board:  
        print(" ".join(row))  
    print()
```

```
def check_q(board, row, col, n):  
    # Check column  
    for i in range(row):  
        if board[i][col] == 'Q':  
            return False
```

Check upper-left diagonal

i, j = row, col

while i > 0 and j >= 0:

if board[i][j] == 'Q':

return False

j -= 1

i -= 1

Check upper-right diagonal

i, j = row, col

```

while i >= 0 and j < n:
    if board[i][j] == 'Q':
        return False
    i -= 1
    j += 1
return True

```

```

def solve_queens(board, row, n):
    if row == n:
        print board
        return True
    for col in range(n):
        if check_q(board, row, col, n):
            board[row][col] = 'Q'
            if solve_queens(board, row + 1, n):
                return True
            board[row][col] = '.'
    return False

```

```

def queens():
    n = int(input("Enter value of N: "))
    board = []
    for i in range(n):
        row = []
        for j in range(n):
            row.append(' ')
        board.append(row)
    if not solve_queens(board, 0, n):
        print("No Solution found")

```

queens()

Q)

Tower of Hanoi:

Herein, we've to move a stack of three disk from a rod to final rod with the help of an alternative rod.

Code:

```
def tower(n, source, auxiliary, destination):
```

```
    if n == 1:
        print("Move disk 1 from", source, "to",
              destination)
        return
```

```
    tower(n-1, source, destination, auxiliary)
```

```
    print("Move disk", n, "from", source, "to", destination)
```

```
    tower(n-1, auxiliary, source, destination)
```

```
n = int(input("Enter number of disks: "))
```

```
tower(n, "A", "B", "C")
```

Output:-

1) For N=4

Enter value of N: . Q..

... Q

Q...

.. Q

For $N = 8$:-

Enter value of N : $Q \dots \dots \dots$
 $\dots \dots Q \dots \dots$
 $\dots \dots \dots Q$
 $\dots \dots Q \dots \dots$
 $\dots Q \dots \dots$
 $\dots \dots \dots Q$
 $\dots Q \dots \dots$
 $\dots Q \dots \dots$

2) for $n = 3$

Enter number of disks : 3

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

Sr No: Date:

Topic:

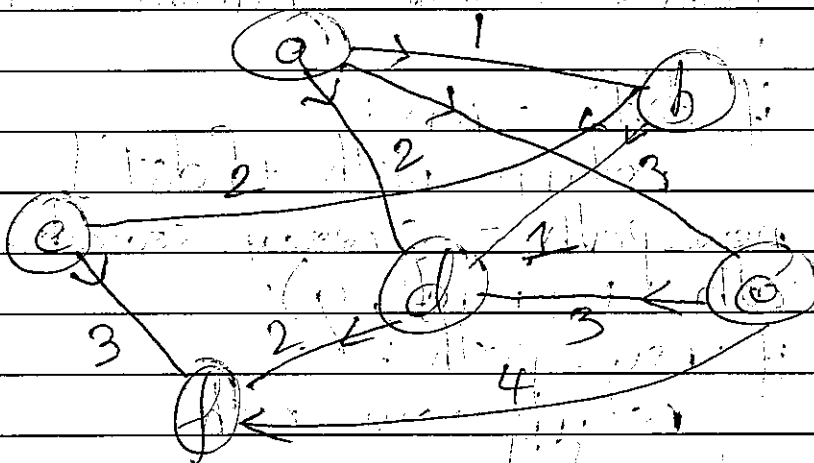
Sign:

4 . 29/6/26

A* Algorithm, Greedy
Best First Search

OM

1) A* Algorithm



Code:-

```

graph = {
    "a": ({ "b": 1, "d": 2, "e": 3 }, 4),
    "b": ({ "c": 2, "d": 1 }, 3),
    "c": ({ "f": 3 }, 2),
    "d": ({ "f": 2 }, 3),
    "e": ({ "d": 3, "f": 4 }, 2),
    "f": ({ }, 0)
}
  
```

```

def a_star(graph, prev, dst, path, pcost, g):
    print("connected nodes of current node",
          prev, "with h(n) values:")
    for n in graph[A][0]:
        if n not in path:
  
```

```

graph[n][n] q[n] = (graph[n][n], graph[prev]
{0}[n])
print(n; "→", q[n])
while q:
    mn = min(q, key=q.get);
    print("taking minimum h(n) vertex: ", mn)
    if dist == min:
        return path + [dst]
    new_path = greedy_search_rec(graph,
mn, dst, path + [mn], q)
    if new_path != 2:
        return new_path
    return []

```

```

source = input("Enter source vertex: ")
dest = input("Enter destination vertex: ")
path = a_star(graph, source, dest, [source])
if path:
    print(path)
else:
    print("path not found.")

```

Output :-

For the graph:

pto →

$a \rightarrow b(1)$

$a \rightarrow d(2)$

$a \rightarrow e(3)$

$b \rightarrow c(2)$

$b \rightarrow d(1)$

$e \rightarrow f(3)$

$d \rightarrow f(2)$

$e \rightarrow d(3)$

$e \rightarrow f(4)$

With heuristic values:

$h(a) = 4$

$h(b) = 3$

$h(c) = 2$

$h(d) = 3$

$h(e) = 2$

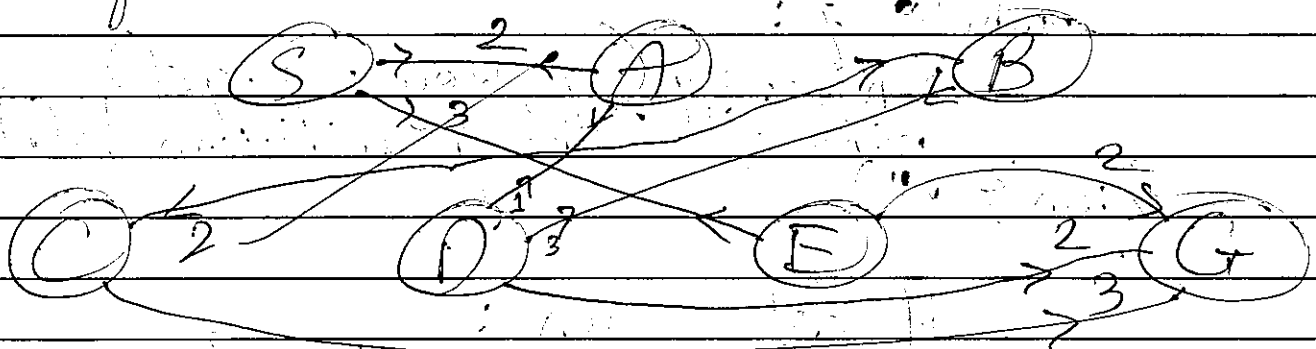
$h(f) = 0$

Shortest path: $[a, d, f]$

Total cost: $2 + 2 = 4$

2)

Diagram:



Code:

graph = {

'S': ({'A': 2, 'E': 3}, 6),

'A': ({'S': 2, 'D': 1}, 3),

'B': ({'C': 2, 'D': 3}, 2),

'C': ({'B': 2, 'G': 3}, 2),

'D': ({'A': 1, 'B': 3, 'G': 2}, 4),

```

'E': ({ 'S': 3, 'G': 2 }, 5),
'G': ({ }, 0)
}

```

```

def greedy-search-rec (graph, prev, dst, path, q):
    # access neighbours from the first
    # element of the tuple
    neighbours = graph[prev][0].keys()

```

```

    for n in neighbours:
        if n not in path:
            # Access heuristic from the 2nd
            # element of the tuple
            g[n] = graph[n][1]
            print(f"{n} → {g[n]}")

```

```

    while q:
        mn = min(q, key = q.get)
        print(f"Taking minimum h(n) vertex {mn}")

```

```

        if dst == mn:
            return path + [dst]

```

```

        current_node = mn
        del q[mn]

```

```

        new_path = greedy-search-rec (graph,
            current_node, dst, path + [current_node], q)
        if new_path:

```

return new_path

return []

Execution

```
source = input("Enter Source Vertex: ")  
result = greedy_search_rec(graph, source, G,  
                             {source}, { })  
print("Resulting Path: ", result)
```

Output:

Enter Source Vertex: S

A → 3

E → 5

Taking minimum $h(n)$ vertex: A

D → 4

Taking minimum $h(n)$ vertex: D

B → 2

G → 0

Taking minimum $h(n)$ vertex: G

Resulting Path: ['S', 'A', 'D', 'G']

QW

Definition:- Any search algorithm

1) A^* Algorithm: It's a popular informed search algorithm for finding shortest path between a start node & goal node in a graph.

2) Greedy Best First Search: It's an informed search algorithm that chooses node with smallest heuristic value $h(n)$. It tries to reach goal as quickly as possible.

Sr.No.	Date	Topic	Sign
7	31/7/26	Shuffle Deck of Cards and Tic Tac Toe game	<u>DM</u>

a) Shuffle Deck of Cards

Code:-

```
import random
suits = ['Hearts', 'Spades', 'Diamonds', 'Clubs']
ranks = ['2', '3', '4', '5', '6', '7', '8', '9', '10', 'Jack', 'Queen', 'King', 'Ace']
deck = [rank + " of " + suit for suit in suits for rank in ranks]
random.shuffle(deck)

print("Shuffled deck of cards:")
for card in deck:
    print(card)
```

Output:-

Shuffled deck of cards:
Jack of Clubs
2 of Diamonds
Ace of Hearts
King of Clubs
4 of Spades
5 of Diamonds

b) Tic Tac Toe

```
board = [' ' for _ in range(9)]
```

```
player = 'X'
```

```
def show_board():
```

```
    print(f' | {board[0]} | {board[1]} |  
    {board[2]} | ')
```

```
    print(' - - - - - ')  
    print(f' | {board[3]} | {board[4]} |  
    {board[5]} | ')
```

```
    print(' - - - - - ')  
    print(f' | {board[6]} | {board[7]} |  
    {board[8]} | ')  
    print(' - - - - - ')
```

```
def is_winner(p):
```

```
    return (
```

```
        (board[0] == p and board[1] == p and  
        board[2] == p) or
```

```
        (board[3] == p and board[4] == p and  
        board[5] == p) or
```

```
        (board[6] == p and board[7] == p and  
        board[8] == p) or
```

```
        (board[0] == p and board[3] == p and  
        board[6] == p) or
```

```
        (board[1] == p and board[4] == p and  
        board[7] == p) or
```

```
        (board[2] == p and board[5] == p and  
        board[8] == p) or
```

```
        (board[0] == p and board[4] == p and
```

```
board[8] == p) or
(board[2] == p and board[4] == p and
board[6] == p)
```

```
def is_tie():
    return ' ' not in board
```

```
def game():
    global player
    while True:
        show_board()
        try:
            move = int(input(f"Player {player} -
win!") - 1))
            if 0 <= move <= 8 and board[move] == ' ':
```

```
board[move] = player
```

```
if is_winner(player):
```

```
    show_board()
```

```
    print(f"Player {player} win!")
    break
```

```
elif is_tie():
```

```
    show_board()
```

```
    print("It's a tie!")
    break
```

```

if player == 'X':
    player = 'O'
else:
    player = 'X'

```

```

else:
    print("Invalid move")
except (ValueError, IndexError):
    print("Invalid input. Enter
a number 0-8")

```

game()

Output:-

1	1	1	1
1	1	1	1
1	1	1	1

Player X enter 0-8: 2

1	1	1	X
1	1	1	1
1	1	1	1

Player O enter 0-8: 4

1	1	1	X
1	1	O	1
1	1	1	1

Player X enter 0-8: 5

DATE:

1	1	1	X	1	
1	1	0	1	X	1
1	1	1	1	1	1

Player O enter O-8:7

1	1	1	X	1	
1	1	0	1	X	1
1	1	0	1	1	1

Player X enter O-8:8

Player X wins!

DATE:

Sr.No	Date	Topic	Sign
9	21/7/26	Derive Expression based on Associative and Distributive Law	@

A) Associative Law :-
Logical Operator

Code :-

```
print("Enter 1 or 0 for boolean value")
A = bool(int(input("Enter A: ")))
B = bool(int(input("Enter B: ")))
C = bool(int(input("Enter C: ")))
```

AND

LHS = (A and B) and C
RHS = A and (B and C)

```
print("\n Associative Law (AND) :-")
```

```
print("LHS = ", LHS)
```

```
print("RHS = ", RHS)
```

```
if LHS == RHS:
```

```
    print("Associative law for AND is satisfied.")
```

```
else:
```

```
    print("Associative law for AND is not satisfied.")
```

OR

LHS = (A or B) or C

RHS = A or (B or C)

```
if LHS == RHS:
```

```
    print("Associative Law for OR is satisfied.")
```

```
else:
```

```
print("Associative Law for OR is not-satisfied.")
```

B) Distributive Law :-

Logical Operator :-

```
A = bool(input("Enter A: "))  
B = bool(input("Enter B: "))  
C = bool(input("Enter C: "))
```

LHS = A and (B ~~and~~ or C)

RHS = (A and B) or (A and C)

```
print("In A and (B or C)")
```

```
print("LHS: ", LHS)
```

```
print("RHS: ", RHS)
```

```
if LHS == RHS:
```

```
    print("Distributive law is satisfied.")
```

```
else:
```

```
    print("Distributive law is not satisfied.")
```

O/P :-

Enter A: 1

Enter B: 0

Enter C: 1

A and (B ~~and~~ or C)

LHS = True

RHS = True

Distributive Law is satisfied.

Arithmetic Operators :-

```

A = int(input("Enter A : "))
B = int(input("Enter B : "))
C = int(input("Enter C : "))

```

O/P :- Enter 1 OR 0

Enter A : 1

Enter B : 0

Enter C : 1

Associative Law (AND/OR)

LHS = False

LHS = False

Associative law for OR is satisfied.

2. Arithmetic :-

```

A = int(input("Enter A : "))
B = int(input("Enter B : "))
C = int(input("Enter C : "))

```

LHS = $(A + B) + C$

RHS = $A + (B + C)$

if LHS == RHS:

print("Assertive Law for addition satisfied.")

else:

print("Assertive Law for addition not satisfied.")

O/P :- Enter A : 2

Enter B : 5

Enter C : 7

LHS = 14

RHS = 14

Associative Law for addition satisfied.

```
LHS = A * (B + C)
RHS = (A * B) + (A * C)
print("\n Distributive Law")
print("LHS: ", LHS)
print("RHS: ", RHS)
if LHS == RHS:
    print("Distributive Law satisfied")
else:
    print("Distributive Law not satisfied.")
```

O/P:- Enter A : 1
Enter B : 2
Enter C : 3
Distributive Law
LHS = 5
RHS = 5
Distributive Law satisfied.